

Technical Report

알라딘 도서 리뷰 분석 서비스

— 알라딘을 메인으로 크롤링한 리뷰 데이터 기반 LSTM 감성분석 웹 서비스 —

과목	AI서비스개발 (융합 프로젝트 실습 · 웹크롤링)
작성자	박종인
주제	알라딘을 메인으로 크롤링한 도서 리뷰 + LSTM 감성분석 웹 서비스

목차

- 1. 프로젝트 개요 2. 서비스 개요 3. 요구사항 정의 4. 아키텍처 설계
- 5. 주요 기술 6. 테스트 결과 7. 프로젝트 결과 분석 8. 개인별 학습 성과

1.1 프로젝트 주제

알라딘을 메인 대상으로 국내 온라인 서점의 도서 리뷰를 대규모로 직접 크롤링하여(알라딘에서 약 100만 건 수집), 도서 검색·평점 순위·감성 분석을 제공하고, 리뷰 텍스트를 **LSTM 딥러닝 모델**이 읽어 긍정·중립·부정을 판별하는 **Streamlit 웹 서비스**이다. 크롤러는 다른 온라인 서점으로도 확장 가능한 구조로 설계했다.

1.2 프로젝트 선정 이유

평소 웹소설을 즐겨 읽고 종이책도 종종 읽으면서, **책 속의 문장은 실제로 눈에 보이는 것보다 훨씬 자세히 표현되어 있고**, 그 표현을 읽는 사람마다 **서로 다른 관점에서 받아들인다는** 점에 흥미를 느꼈다. 그래서 같은 책이라도 독자마다 다른 감정을 느끼고 다른 생각을 하게 된다. 이런 **'독자마다 다른 감상'이 리뷰에 고스란히 드러난다**고 보았고, 수많은 리뷰를 모아 감성으로 분석하면 한 책에 대한 다양한 반응을 한눈에 볼 수 있겠다고 생각한 것이 이 주제를 선택한 출발점이다.

온라인 서점에는 도서마다 리뷰가 수천~수만 개씩 쌓여 있지만, 이를 한눈에 파악할 방법이 마땅치 않다는 점에도 주목했다. 리뷰는 양이 많아 일일이 읽기 어렵고, 서점이 보여주는 평점 평균만으로는 독자들이 책 내용에 대해 실제로 어떻게 느꼈는지까지 알기 어렵다.

그래서 데이터가 풍부한 **알라딘을 메인**으로 리뷰를 크롤링해 모으고, 감성분석 모델로 리뷰를 자동 분류·요약하면 독자에게 실질적인 도움이 되겠다고 판단해 주제로 선정했다.

아울러 **웹 크롤링 → 데이터 전처리 → 딥러닝 학습 → 웹 서비스 구현**으로 이어지는 전 과정을 직접 경험하기에 적합한 주제였다.

1.3 개발 환경

사용 언어	Python 3.10
개발 도구	Jupyter Notebook(모델 학습·실험), VS Code(앱 개발), Anaconda 가상환경(aiservice26)
주요 라이브러리	Streamlit, pandas, TensorFlow/Keras, KoNLPy(Okt), matplotlib, joblib, requests
모델	LSTM(순환신경망) 기반 3분류 감성분석 모델
실행 방식	로컬 웹앱 — 터미널에서 streamlit run app.py 실행 → 브라우저(localhost:8501) 접속. KoNLPy 사용을 위해 JDK(Java) 설치 필요

2.1 서비스 설명

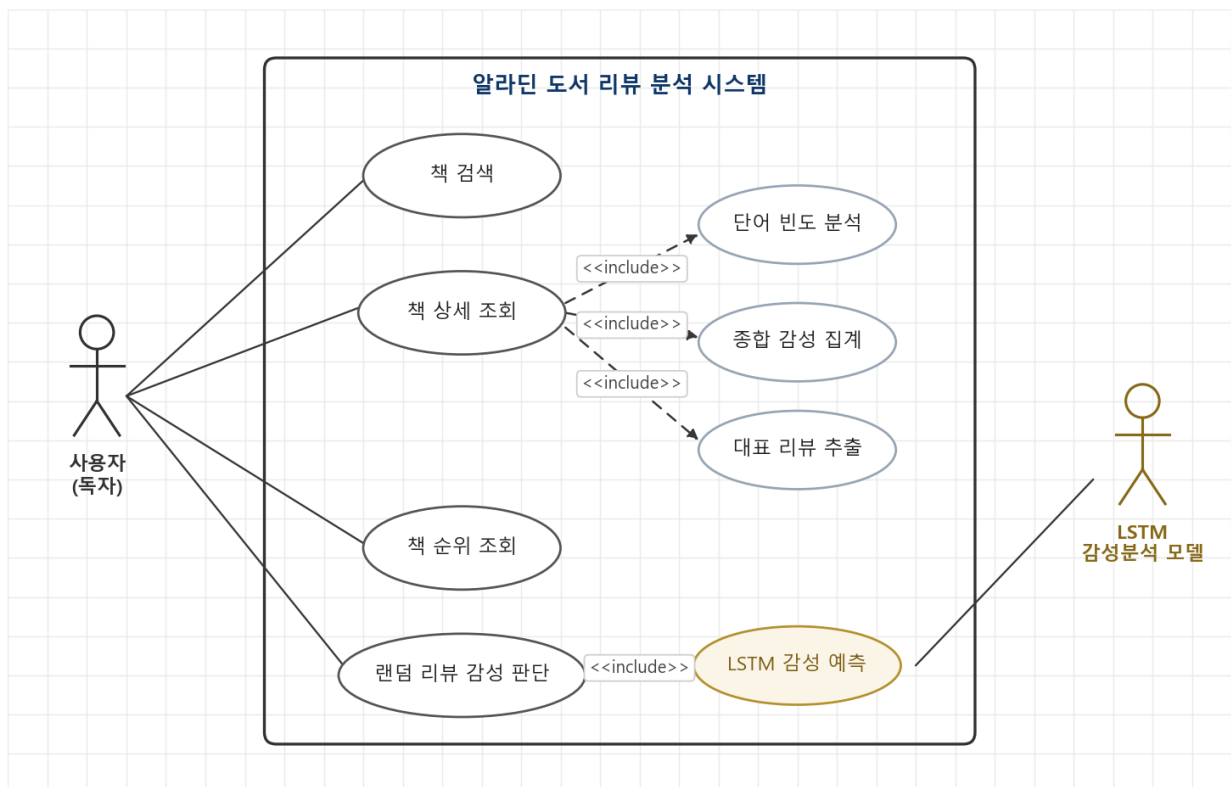
핵심 기능: 사용자가 책 제목의 일부만 입력해도 관련 도서를 찾아, 해당 도서의 리뷰를 종합해 종합 감성(긍정·중립·부정 비율)·평균 평점·대표 리뷰·자주 나온 단어를 한 화면에 요약해 준다. 또한 임의의 리뷰를 뽑아 학습된 LSTM 모델이 내용만 보고 감성을 판별하고, 이를 실제 별점과 비교해 보여준다.

주요 사용자와 사용 상황

- **일반 독자(구매 예정자)** — 책을 사기 전, 별점만으로는 알기 어려운 실제 평판을 빠르게 확인하고 싶을 때.
- **다독자·리뷰 탐색자** — 평점 높은/낮은 책, 리뷰가 많은 화제작을 순위로 훑어보고 싶을 때.
- **데이터 관점 사용자** — 별점과 리뷰 내용이 어긋나는 사례(예: 별점 5점인데 내용은 부정)를 확인하고 싶을 때.

2.2 주요 기능

■ Usecase Diagram



사용자(독자)는 책 검색·책 상세 조회·책 순위 조회·랜덤 리뷰 감성 판단을 수행하며, 책 상세 조회는 종합 감성 집계·단어 빈도 분석·대표 리뷰 추출을 <<include>> 관계로 포함한다. 랜덤 리뷰 감성 판단은 LSTM 감성 예측을 포함하며, 이 예측은 학습된 LSTM 감성분석 모델이 수행한다.

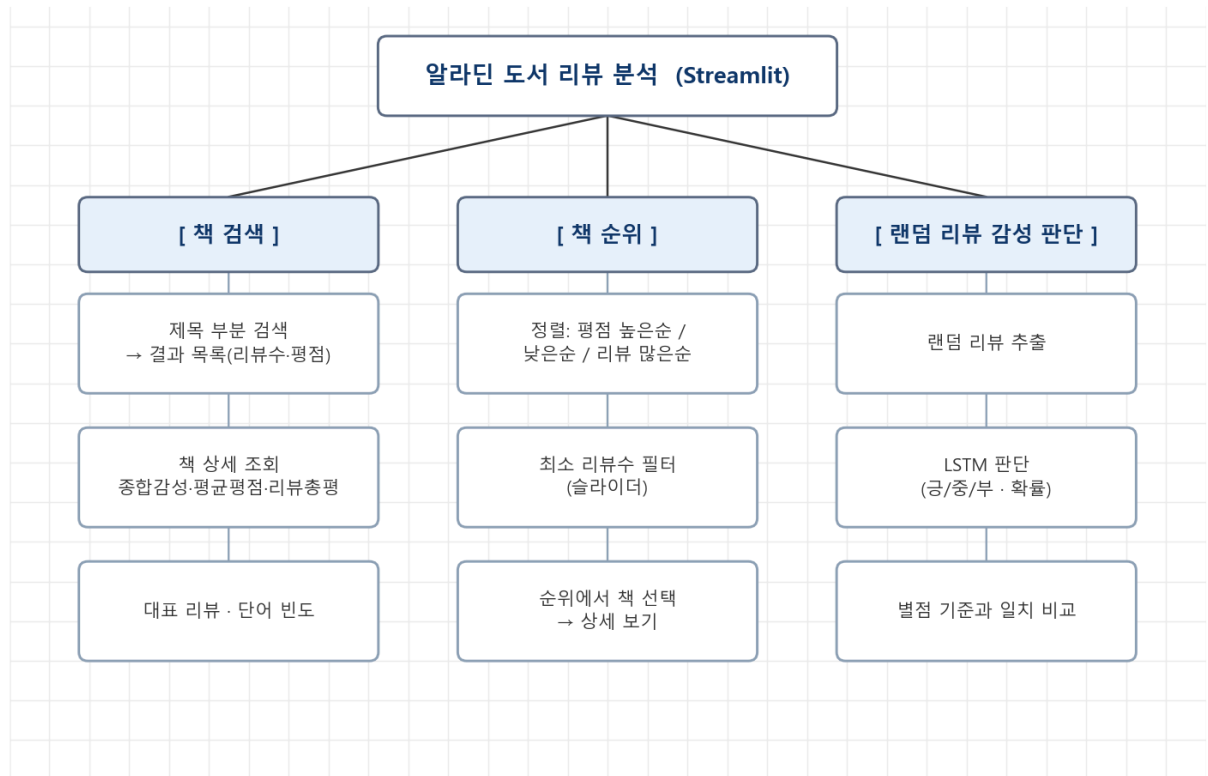
■ 기능별 설명

기능	설명
책 검색	제목 일부 입력 시 포함되는 모든 도서를 리뷰수 순으로 검색
책 상세 / 종합 감성	선택 도서의 리뷰수·평균평점, 긍정/중립/부정 비율 막대, 리뷰 총평(요약 문장)
대표 리뷰	긍정·부정 대표 리뷰를 자동 선별해 인용 표시
단어 빈도 분석	Okt 형태소 분석으로 명사·형용사·동사 빈출 단어 Top-N 막대그래프
책 순위	평점 높은순/낮은순/리뷰 많은순 랭킹 + 최소 리뷰수 필터
랜덤 리뷰 감성 판단	랜덤 리뷰를 LSTM이 긍/중/부 판별(확신도·확률 제시) 후 별점 기준과 일치 여부 비교

■ 사용자 유형 설명

본 서비스는 단일 사용자 유형(일반 사용자/독자)을 대상으로 하며 별도 로그인·권한 구분이 없다. 누구나 검색·조회·감성판단 기능을 동일하게 사용한다.

2.3 메뉴 구성도



앱은 세 개의 탭(책 검색 · 책 순위 · 랜덤 리뷰 감성 판단)으로 구성되며, 각 탭은 위와 같은 하위 기능으로 이어진다.

2.4 E2E 사용자 시나리오

가상의 사용자 '홍길동'이 앱을 처음 사용하는 전체 흐름(End-to-End)이다.

1. 진입 및 준비

홍길동은 터미널에서 **streamlit run app.py**를 실행해 앱에 접속한다.

앱이 처음 켜지며 **data/filtered/**의 CSV 26개 조각을 자동 병합해 약 100만 건의 리뷰를 메모리에 올린다(최초 1회, 이후 캐시).

메인 화면에 **책 검색** · **책 순위** · **랜덤 리뷰 감성 판단** 세 개의 탭이 나타난다.

2. 검색 및 탐색

홍길동은 [책 검색] 탭에서 관심 있는 책 제목의 일부 "**사피엔스**"를 입력한다.

제목에 '사피엔스'가 포함된 도서 목록이 리뷰수 순으로 나타나고, 홍길동은 그중 한 권을 선택한다. 선택한 책의 **리뷰수** · **평균 평점** · **긍정/중립/부정 비율** · **리뷰 총평** · **대표 리뷰** · **단어 빈도**를 한 화면에서 확인한다.

이어 [책 순위] 탭으로 이동해 "**평점 높은순**"으로 정렬하고 최소 리뷰수를 50으로 맞춰, 리뷰가 충분한 상위 도서들을 훑어본다.

3. 감성 판단 및 마무리

홍길동은 [랜덤 리뷰 감성 판단] 탭에서 "**랜덤 리뷰 뽑기**"를 누른다.

학습된 **LSTM 모델**이 뽑힌 리뷰의 **내용만** 보고 긍정·중립·부정을 판단하고, 확신도·확률과 함께 **실제 별점 기준과 일치 여부**를 보여준다.

홍길동은 별점과 내용이 어긋나는 리뷰(예: 별점 5점인데 내용은 부정) 사례를 확인하며, 별점만으로는 알기 어려운 실제 반응을 파악하고 앱 사용을 마친다.

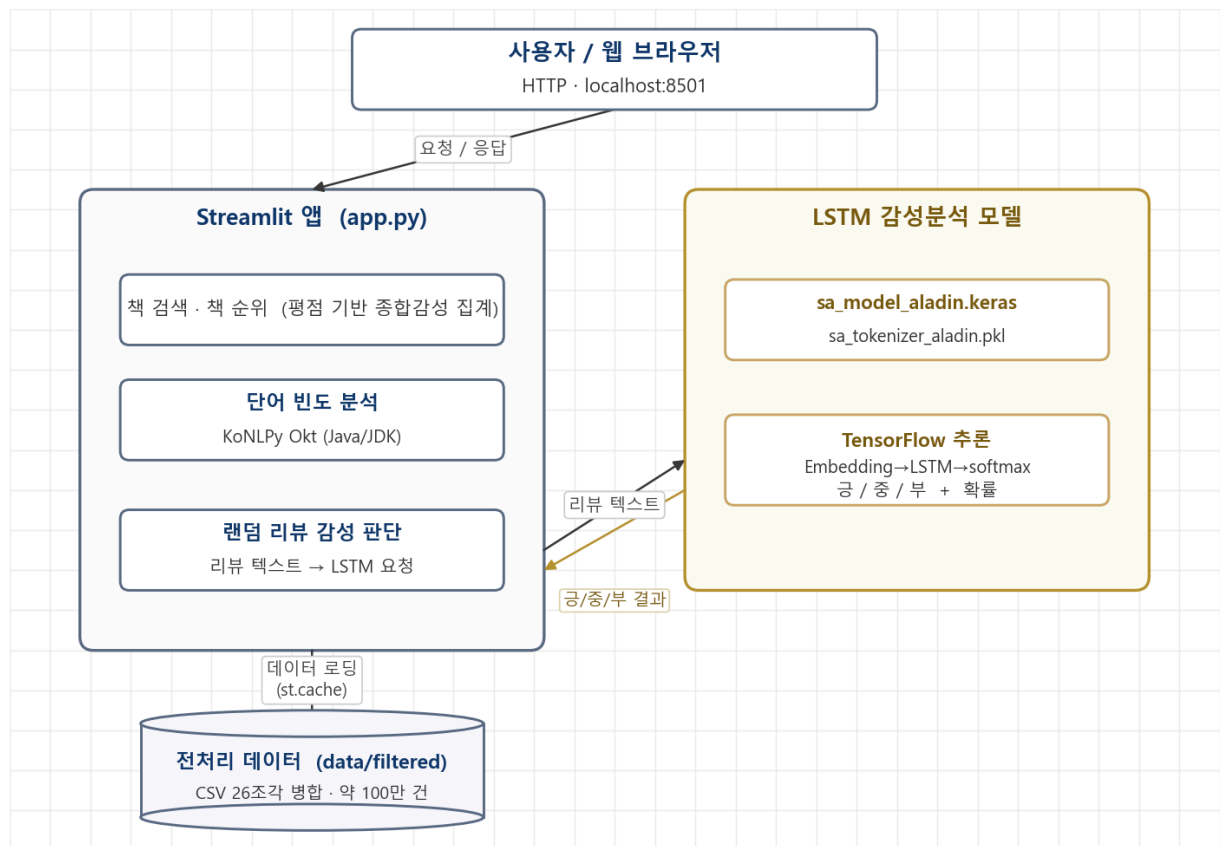
3.1 기능 요구사항

ID	요구사항	상세
FR-01	도서 검색	제목 일부 입력으로 도서를 검색할 수 있어야 한다
FR-02	종합 감성 제공	도서별 리뷰의 긍정/중립/부정 비율과 평균 평점을 제공한다
FR-03	리뷰 총평·대표리뷰	리뷰를 요약한 총평 문장과 대표 긍정/부정 리뷰를 제시한다
FR-04	단어 빈도 분석	도서 리뷰의 빈출 단어를 형태소 분석해 시각화한다
FR-05	도서 순위	평점·리뷰수 기준으로 도서를 정렬·조회할 수 있어야 한다
FR-06	텍스트 감성 판별	리뷰 텍스트를 LSTM 모델이 긍/중/부로 분류하고 별점과 비교한다

3.2 비기능 요구사항

ID	항목	내용
NFR-01	성능	100만 건 데이터를 캐싱해 재조회 시 지연 없이 응답
NFR-02	사용성	제목 일부만 입력해도 검색되는 직관적 UI(탭 구성)
NFR-03	안정성	데이터/모델 로딩 실패, 감성판단 예외 시 오류 메시지로 안전 처리
NFR-04	확장성	데이터를 분할 저장해 대용량에도 대응, 서점 추가 확장 가능한 구조
NFR-05	이식성	모든 경로를 상대경로로 처리해 클론 후 바로 실행 가능

4.1 시스템 구성도



브라우저 요청은 **Streamlit 앱(app.py)**이 처리한다. 검색·순위·종합감성은 전처리 CSV(약 100만 건)를 캐시해 집계하고, 랜덤 리뷰 감성 판단만 **LSTM 모델(TensorFlow)**에 리뷰 텍스트를 보내 금/중/부를 예측받는다. 단어 빈도 분석은 KoNLPy(Okt) 형태소 분석을 사용한다.

4.2 프로젝트 파일 구조

book-webcrawling/

- ├── **01_source_code_and_data/** # 소스코드 + 원본(크롤링) 데이터
 - ├── crawler/ # 알라딘 리뷰 크롤러(v2) — 100만 건 수집, 서점 확장 가능 구조
 - ├── preprocessing/ # 전처리 스크립트
 - ├── notebooks/ # 학습·실습 노트북
 - └── data/ # raw 크롤링 CSV(part01~28)
- ├── **02_executable_and_model/** # 실행앱 + 전처리·학습 데이터 + 모델
 - ├── app.py # Streamlit 실행 파일 + requirements.txt
 - ├── data/filtered/ (26조각) # 전처리 데이터(앱 사용)
 - ├── data/ing/ (9조각) # 학습용 데이터(라벨·토큰화)
 - ├── model/ # LSTM 모델 + 토크나이저
 - └── eval/ # 학습로그 · 평가결과(json)
- └── **03_docs/** # 발표자료 · Technical Report · 시연영상

기술	사용 목적 및 설명
웹 크롤링 (requests, 비동기)	알라딘을 메인 대상으로 도서·리뷰 페이지에서 제목·리뷰내용·평점 등을 약 102만 건 수집(비동기 크롤러로 수집 속도 향상). 크롤러는 다른 온라인 서점으로도 확장 가능하도록 구조를 일반화
데이터 전처리 (pandas)	8컬럼 원본을 3컬럼으로 정리, 배송·굿즈 등 무관 리뷰를 키워드로 제거, 평점→감성 라벨링 및 3클래스 균형 샘플링
형태소 분석 (KoNLPy Okt)	한국어 리뷰를 토큰화(형태소 단위)하여 LSTM 입력과 단어 빈도 분석에 사용. Java(JDK) 기반
LSTM 감성분석 (TensorFlow/ Keras)	Embedding → LSTM → Dense(tanh) → Dense(softmax) 구조의 순환신경망으로 리뷰 텍스트를 긍/중/부 3분류. 시퀀스 max_len=100 패딩, RMSprop(lr=0.001)
웹 서비스 (Streamlit)	파이썬만으로 데이터 앱을 구현. st.cache_data/cache_resource로 대용량 로딩·모델 추론 최적화, matplotlib로 그래프 시각화

6.1 테스트 계획

테스트 항목: 도서 검색, 종합감성 표시, 순위 정렬, 단어빈도 분석, LSTM 감성판단, 예외 처리

테스트 조건: 전처리 데이터(100만 건) 로딩 완료 상태에서 각 탭 기능을 실제 입력으로 실행하여 결과를 육안 검증

6.2 테스트 케이스

TC	입력/시나리오	기대 결과	결과
TC-01	검색창에 제목 일부 입력	포함 도서 목록이 리뷰수순 표시	통과
TC-02	존재하지 않는 제목 검색	"검색 결과가 없습니다" 안내	통과
TC-03	도서 선택	종합감성·평균평점·총평·대표리뷰 표시	통과
TC-04	순위 정렬 3종 전환	평점/리뷰수 기준으로 랭킹 재정렬	통과
TC-05	단어 빈도 분석 체크	빈출 단어 Top-N 막대그래프 출력	통과
TC-06	랜덤 리뷰 감성 판단	LSTM 판단(긍/중/부·확률) + 별점 비교	통과

6.3 오류 처리 결과

상황	처리 방식	결과
데이터 파일 없음	st.error 후 st.stop()으로 안전 종료	앱 중단 없이 안내
모델 추론 중 예외	try/except로 감싸 오류 메시지 출력	앱 계속 동작
한글 폰트 부재	except로 무시하고 기본 폰트 사용	그래프 정상 출력
결측·비정상 평점	전처리 단계에서 dropna·평점 1~5 범위 필터	노이즈 제거

7.1 구현 완료 기능

- 알라딘을 메인으로 도서 리뷰 약 100만 건 크롤링 및 전처리 완료(무관 리뷰 제거, 3클래스 균형 데이터 구축)
- 도서 검색 · 종합감성 · 리뷰총평 · 대표리뷰 · 단어빈도 · 도서순위 기능 구현
- LSTM 감성분석 모델 학습 및 앱 연동 — 랜덤 리뷰 감성 판단(별점 비교) 기능 구현

모델 성능 (테스트셋 7,741건, 3클래스 균형)

지표	값	클래스	F1
정확도(Accuracy)	62.6%	부정	0.71
macro-F1	0.618	중립	0.47
손실(loss)	0.82	긍정	0.68

7.2 구현하지 못한 기능

- AI 텍스트 감성 일괄 분석: 도서별 전체 리뷰를 모델로 재분류하는 기능은 100만 건 추론 비용 문제로 제외하고 평점 기반 집계로 대체.
- 모델 구조·시각화: 양방향 LSTM 등 개선된 구조 미적용, 워드클라우드·기간별 트렌드 시각화 미완.

7.3 어려웠던 점과 해결 방법

어려웠던 점	해결 방법
100만 건 데이터 로딩이 매우 느림	st.cache_data로 캐싱하여 최초 1회만 로딩, 이후 즉시 응답
배송·굿즈 등 책과 무관한 리뷰가 감성 왜곡	무관 키워드(배송·특전·품질 등) 필터로 전처리 단계에서 제거
긍정 리뷰 92% 편향으로 모델이 긍정만 예측	3클래스를 각 25,806건으로 균형 샘플링하여 재학습
KoNLPy가 Java 의존성으로 실행 오류	JDK 설치 + 환경변수 설정, 전용 conda 환경(aiservice26)으로 분리

8.1 새롭게 배우거나 성장한 부분

- 웹 크롤링으로 대규모 실데이터(100만 건)를 직접 수집하는 전 과정을 경험했다.
- 대용량 데이터의 정제·라벨링·균형 샘플링 등 전처리가 모델 성능에 결정적임을 체감했다.
- LSTM(순환신경망)으로 한국어 텍스트 감성분석 모델을 학습·평가·저장하고 실제 앱에 연동해 보았다.
- Streamlit으로 데이터·모델을 실제 사용할 수 있는 웹 서비스로 배포하는 흐름(캐싱·UI 구성)을 익혔다.
- 수집→전처리→학습→서비스로 이어지는 end-to-end 파이프라인을 스스로 완주했다.

8.2 아쉬운 부분과 개선 방향

- 모델 정확도(62.6%)와 중립 분류(F1 0.47)가 아쉬움 — 평점을 정답으로 삼아 라벨 노이즈가 컸다.
→ 데이터 정제와 하이퍼파라미터·모델 구조 개선(양방향 LSTM, 임베딩·시퀀스 길이 튜닝 등)으로 내가 학습시킨 LSTM 모델의 정확도를 더 끌어올리고 싶다.
- 이번 프로젝트는 알라딘 리뷰를 중심으로 사용했는데, 앞으로 다른 서점의 데이터를 추가로 모아 더 다양한 사이트의 리뷰를 함께 분석해보고 싶다.
- 기능을 넓히기보다 완성도(오류 처리·시각화)를 더 다졌으면 하는 아쉬움이 있다.

첨부 — 발표자료(웹크롤링프로젝트_발표_박종인.pptx) · 시연영상(웹크롤링_시연영상_최종본.mp4) · 실행방법(02_executable_and_model/README.md)